

③ This problem will apply algorithm unrolling to structured dictionary learning. Assume data points $y_1, \dots, y_n \in \mathbb{R}^d$. For a data point $y \in \mathbb{R}^d$ we consider the optimization problem

$$\min_{x \in \Delta^m} \|y - Ax\|_2^2 + \lambda \sum_{j=1}^m x_j \|y - a_j\|_2^2$$

where $A = [a_1, \dots, a_m] \in \mathbb{R}^{d \times m}$ is a learnable dictionary, $x \in \mathbb{R}^m$ is constrained to the probability simplex $\Delta^m := \{x \in \mathbb{R}^m : x_j \geq 0, \mathbf{1}^T x = 1\}$ and $\lambda > 0$ is a regularization parameter. Given $y_1, \dots, y_n \in \mathbb{R}^d$, we want to learn the dictionary A and sparse coefficients $x_1, \dots, x_m \in \mathbb{R}$.

(a) Provide a brief interpretation of the dictionary learning problem. What kind of structure do we impose on the representation of a data point using the atoms? What happens as $\lambda \rightarrow \infty$?

Soln: In words, minimizing $\|y - Ax\|_2^2$ for a data point $y \in \mathbb{R}^d$ and solution $x \in \Delta^m$ means that we want to find the best approximation to y on the probability simplex Δ^m . On the other hand, the regularization term $\sum_{j=1}^m x_j \|y - a_j\|_2^2$

can be rewritten as follows:

$$\begin{aligned} \sum_{j=1}^m x_j \|y - a_j\|_2^2 &= \sum_{j=1}^m x_j (\|y\|_2^2 - 2y^T a_j + \|a_j\|_2^2) \\ &= \|y\|_2^2 \sum_{j=1}^m x_j - 2y^T \sum_{j=1}^m x_j a_j + \sum_{j=1}^m x_j \|a_j\|_2^2 \\ &= \|y\|_2^2 - 2y^T \sum_{j=1}^m x_j a_j + \sum_{j=1}^m x_j \|a_j\|_2^2 \\ &= y^T y - 2y^T \sum_{j=1}^m x_j a_j + \sum_{j=1}^m x_j \|a_j\|_2^2 \end{aligned}$$

So it follows that

$$\begin{aligned} \nabla_y \left(\sum_{j=1}^m x_j \|y - a_j\|_2^2 \right) &= 2y - 2 \sum_{j=1}^m x_j a_j \\ \Rightarrow \nabla_y (\cdot) &= 0 \text{ iff } y = \sum_{j=1}^m x_j a_j \end{aligned}$$

In words, for a fixed dictionary and coding $\bar{x}$, the regularization term is minimized for a data point $y \in \mathbb{R}^d$ for which $y \approx \sum_{j=1}^m \bar{x}_j a_j$, i.e. y can be written as a convex combination of the atoms a_j . Thus, we want to learn anchors a_j which lie on or near $\text{conv}(\{y_i\})$.

Furthermore, by minimizing $\lambda x_j \|y - a_j\|_2^2$ for each j , we can penalize large $\|y - a_j\|_2^2$ using large λ . So, as $\lambda \rightarrow \infty$, we should expect the columns of A to contain extreme points of $\text{conv}(\{y_i\})$, thus allowing us to exactly reconstruct each y_i as $\sum_{j=1}^m x_j a_j$.

(c) Let $T := \{x \in \mathbb{R}^m : \sum_{i=1}^m x_i = 1\}$. Show that orthogonally projecting $z \in \mathbb{R}^m$ onto T has the form $\tilde{z}_i = z_i - \tau$, and derive the value of τ . Provide a geometric explanation for why the projection direction must lie along the normal vector of T .

E: This is just an orthogonal projection onto an affine hyperplane. To make this more clear, recall that for a vector space V with a subspace W and $v \in V$ that is nonzero, an affine space is of the form $A = v + W$. Observe that for any $b \in \mathbb{R}^m$ at $b^T \tilde{\tau} = 0$ we have

$$T = b + V \text{ where } V = \{x \in \mathbb{R}^m : x^T \tilde{\tau} = 0\}.$$

We can also write this in another form:

$$T = \{x \in \mathbb{R}^m : \exists c \in \mathbb{R}^{m-1} : x = Mc + b\}$$

where $b^T \tilde{\tau} = 0$ and the columns of $M \in \mathbb{R}^{m \times (m-1)}$ span $V = \mathbb{R}^m$. This is well-defined since V is in fact a linear subspace (see Q2 part b where I provide a basis). This formulation is useful, since we now have

$$\tilde{z} = \min_{x \in T} \|z - x\|_2^2 = \min_{c \in \mathbb{R}^{m-1}} \|z - (Mc + b)\|_2^2.$$

We expand and minimize:

$$\begin{aligned} \|z - (Mc + b)\|_2^2 &= (z - (Mc + b))^T (z - (Mc + b)) \\ &= z^T z - 2z^T (Mc + b) + (Mc + b)^T (Mc + b) \\ &= z^T z - 2c^T M^T z - 2z^T b + c^T M^T M c + 2c^T M^T b + b^T b \\ \Rightarrow \nabla_c (\|z - (Mc + b)\|_2^2) &= -2M^T z + 2M^T b + 2M^T M c \\ &= -2M^T (z - b) + 2M^T M c \end{aligned}$$

$$\Rightarrow \nabla_c (\cdot) = 0 \text{ iff } c_* = (M^T M)^{-1} M^T (z - b) = M^+ (z - b)$$

This is well-defined since M is full-rank, which also implies that $\exists x_* \in T$ which is unique and satisfies $x_* = Mc_* + b$.

$$\therefore \text{We have } \tilde{z} = b + MM^+ (z - b).$$

But T is a hyperplane, so we can simplify a bit. For a general hyperplane

$$T_w = \{x \in \mathbb{R}^m : w^T x = \alpha\} = w^\perp,$$

then $b \in T_w \Rightarrow b = \frac{\alpha}{w^T w} w$, which is an orthogonal projection in the direction of w . Note that this together with the fact that $T_w = w^\perp$ is why the projection direction lies along the normal vector to T_w ; that is they are parallel.

Now we can write $\hat{z} = \frac{\alpha}{w^T w} w + MM^T (z - \frac{\alpha}{w^T w} w)$

But MM^T is the orthogonal projector onto $w^\perp$, so

$$MM^T = I - ww^T = I - \frac{ww^T}{w^T w}.$$

Now plug-in and simplify:

$$\begin{aligned}\hat{z} &= \frac{\alpha}{w^T w} w + (I - \frac{ww^T}{w^T w}) (z - \frac{\alpha}{w^T w} w) \\ &= \frac{\alpha}{w^T w} w + z - \frac{\alpha}{w^T w} w - \frac{ww^T}{w^T w} z + \frac{ww^T}{w^T w} \frac{\alpha}{w^T w} w \\ &= z - \frac{ww^T}{w^T w} z + \frac{ww^T}{w^T w} \frac{\alpha}{w^T w} w \\ &= z - \frac{ww^T}{w^T w} z + \frac{\alpha}{w^T w} w \\ &= z - \frac{1}{w^T w} (ww^T z - \alpha w)\end{aligned}$$

In this case, $\alpha=1$ and $w=\vec{1}$, so $\frac{1}{w^T w} (ww^T z - \alpha w) = \frac{1}{n} (\vec{1}^T z - \vec{1})$

$$\Rightarrow \tau = \frac{1}{n} (\sum_{i=1}^n z_i - 1) \quad \square$$

c) Consider the Euclidean projection $\Pi_{\Delta^m}(z) = \arg \min_{p \in \Delta^m} \frac{1}{2} \|p - z\|_2^2$.

Using the intuition from the affine projection in (b), argue that the solution must take the form $\max(z_i - \tau, 0)$.

Pf: Note that $\Delta^m := \{x \in \mathbb{R}^m : x_j \geq 0, \vec{1}^T x = 1\}$ and $\mathbb{T} := \{r \in \mathbb{R}^m : \vec{1}^T r = 1\}$, so it follows that $\Delta^m = \{x \in \mathbb{T} : x_j \geq 0\}$. Also, both the projection in (b) and $\Pi_{\Delta^m}(z)$ are unconstrained least-squares problems. Thus,

$$\Pi_{\Delta^m}(z) = \arg \min_{p \in \Delta^m} \frac{1}{2} \|p - z\|_2^2 \in \text{span}(\max(\arg \min_{p \in \mathbb{T}} \frac{1}{2} \|p - z\|_2^2, \vec{0}))$$

Where we apply max entrywise to enforce nonnegativity constraints, ensuring that $\Pi_{\Delta^m}(z) \in \Delta^m \cap \mathbb{T}$. So, from (b) we see that entries of $\Pi_{\Delta^m}(z)$ will be of the form $\max(z_i - \tau, 0)$.

Note that $\Pi_{\Delta^m}(z)$ lies in the span of this vector since removing negative entries of $z - \tau \vec{1}$ fixes the nonnegativity constraint, but may push $z - \tau \vec{1} \notin \mathbb{T}$. In the next part we prove a form for τ which resolves this.

c) Assume that entries of $z \in \mathbb{R}^m$ are sorted in decreasing order and define the active set $S = \{i : z_i - \tau > 0\}$. Show that

$$\tau = \frac{\sum_{i \in S} z_i - 1}{|S|}$$

Notation: Let $(Z - \tau \mathbf{1})_+$ be the vector s.t. $(Z - \tau \mathbf{1})_i = z_i - \tau$ for $i \in S$, and $(Z - \tau \mathbf{1})_i = 0$ otherwise.

Pf: The solution to the Euclidean projection in (c) can be approximated by projection onto $\mathbb{T}$ where we replace all negative coefficients of the solution with zero. The active set S holds all indices $i \in [m]$ for which $z_i - \tau > 0$. These are all of the coefficients which we DO NOT replace.

In the simplest case, $S = [m]$ and we have $\tau = \frac{1}{m}(\sum_{i=1}^m z_i - 1)$ as detailed in part (b).

Assume now that $S \subset [m]$. Since τ is constant and entries of $Z \in \mathbb{R}^m$ are sorted in decreasing order, $\exists k \in \mathbb{Z}_+$ for which $k < m$ and $S = [k]$. Referring back to (b), we can write

$$Z - \frac{1}{m}(\mathbf{1}^T Z - 1) \mathbf{1} = Z - \tau \mathbf{1} \Rightarrow (Z - \tau \mathbf{1})_i = z_i - \tau$$

To enforce $Z - \tau \mathbf{1} \in \mathbb{T}$, it follows that

$$1 = \mathbf{1}^T (Z - \tau \mathbf{1}) = \sum_{i \in [m]} (z_i - \tau) = \sum_{i \in [k]} (z_i - \tau) + \sum_{i \in [m] \setminus [k]} (z_i - \tau)$$

Where $(z_i - \tau) > 0$ for $i \in [k]$ and $(z_i - \tau) < 0$ for $i \in [m] \setminus [k]$. To enforce $Z - \tau \mathbf{1} \in \Delta^m$, we will ignore the coefficients $(z_i - \tau)$ for $i \in [m] \setminus [k]$. So we conclude that

$$1 = \sum_{i \in [k]} (z_i - \tau) = \sum_{i \in [k]} z_i - k\tau = \sum_{i \in S} z_i - |S| \cdot \tau$$

$$\Rightarrow |S| \cdot \tau = \sum_{i \in S} z_i - 1 \Rightarrow \tau = \frac{\sum_{i \in S} z_i - 1}{|S|} \quad \square$$

Interpretation: we can construct τ given a "candidate" active set $S = [k] \subseteq [m]$. We need to check that $\mathbf{1}^T (Z - \tau \mathbf{1})_+ = 1$. Thus, we increment $k \in [m]$ until we find τ that adequately solves the problem.

(e) Bonus: Write the KKT conditions for simplex projection and explain how they lead to the thresholding structure above.

(f) Briefly compare the simplex projection to the Softmax mapping. What sparsity-inducing property does the simplex projection possess?

Soln: Recall that $\text{softmax}: \mathbb{R}^m \rightarrow \Delta^m$ is defined as $(\text{softmax}(Z))_i = \frac{e^{z_i}}{\sum_{j=1}^m e^{z_j}}$

We want to compare $\text{softmax}(Z)$ and $\Pi_{\Delta^m}(Z)$.

Notice that for any $\tau \in \mathbb{R}$ we have

$$(\text{Softmax}(Z + \tau \mathbf{1}))_i = \frac{e^{z_i + \tau}}{\sum_{j=1}^m e^{z_j + \tau}} = \frac{e^{z_i} e^\tau}{e^\tau \sum_{j=1}^m e^{z_j}} = \frac{e^{z_i}}{\sum_{j=1}^m e^{z_j}} = (\text{Softmax}(Z))_i$$

$$\implies \text{Softmax}(Z + \tau \mathbf{1}) = \text{Softmax}(Z).$$

Suppose now that $\Pi_{\Delta^m}(Z) = (Z - \tau \mathbf{1})_+ \in \Delta^m$. The Interpretation to conclude (d) allows us to use an iterative thresholding method to construct $\Pi_{\Delta^m}(Z)$ to be as sparse as possible. However, as $\phi: \mathbb{R} \rightarrow (0, \infty)$, then $\text{Softmax}(Z)$ is always m -sparse. For instance, observe that

$$(\text{Softmax}(Z - \tau \mathbf{1}))_i = \begin{cases} (\text{Softmax}(Z))_i, & i \in S \\ 1/(\sum_{j \in S} e^{z_j} + (m-k)), & i \notin S \end{cases}$$

for $|S| = k$. What this exemplifies is that any sparsity achieved in computing $\Pi_{\Delta^m}(Z)$ is lost when computing $\text{Softmax}(\Pi_{\Delta^m}(Z))$, despite the fact that we remain on Δ^m .

Consider two specific examples:

• $(\text{Softmax}(\vec{0}))_i = \frac{1}{m} \mathbf{1}$; an m -sparse vector from a 0 -sparse vector

⊛ Fix $Z = [\vec{1}, -\vec{1}] \in \mathbb{R}^{2m}$ for $\vec{1} \in \mathbb{R}^m$. Computing $\Pi_{\Delta^m}(Z)$, we choose an active set $S = [m]$ and see that

$$\rightarrow \tau = \frac{m-1}{m} \implies z_i - \tau = 1 - \frac{m-1}{m} > 0 \text{ for } i \in S$$

$$z_i - \tau = -(1 + \frac{m-1}{m}) < 0 \text{ for } i \notin S$$

$$\implies z_m - \tau \geq 0 \geq z_{m+1} - \tau$$

$$\rightarrow \text{It follows that } \Pi_{\Delta^m}(Z) = [\vec{1} - \frac{m-1}{m} \vec{1}, \vec{0}] \text{ for } \vec{1}, \vec{0} \in \mathbb{R}^m$$

$$\rightarrow \text{Observe that } \vec{1}^T (\vec{1} - \frac{m-1}{m} \vec{1}) = m - \frac{m-1}{m} m = 1 \text{ as desired.}$$

So Softmax always maps to a dense vector in $\text{int}(\Delta^m)$, while $\Pi_{\Delta^m}(Z)$ induces sparsity by essentially truncating negative coefficients. □

(g) Fix A and define $f: \mathbb{R}^n \rightarrow \mathbb{R}$ st

$$f(x) = \|y - Ax\|_2^2 + \lambda \sum_{j=1}^m x_j \|y - a_j\|_2^2.$$

Show that f is convex in x , and compute $\nabla_x f(x)$.

Pf: Define $d \in \mathbb{R}^n$ to be the vector st $d_j = \|y - a_j\|_2^2$. Then

$$\nabla_x f(x) = \nabla_x (\|y - Ax\|_2^2 + \lambda x^T d) = 2A^T A x - 2A^T y + \lambda d = 2A^T (Ax - y) + \lambda d$$

Recall that for $A \subseteq \mathbb{R}^n$ a convex set, we say that $g: A \rightarrow \mathbb{R}$ is convex when $\forall x_1, x_2 \in A$ and $\theta \in [0, 1]$ we have

$$g(\theta x_1 + (1-\theta)x_2) \leq \theta g(x_1) + (1-\theta)g(x_2).$$

In fact, this definition is equivalent to the following:

$$g(x_2) \geq g(x_1) + \nabla g(x_1)^T (x_2 - x_1) \text{ for all } x_1, x_2 \in A.$$

For this specific example, pick $x_1, x_2 \in \mathbb{R}^m$ and observe that

$$\begin{aligned} f(x_2) - f(x_1) &= \|y - Ax_2\|_2^2 + \lambda x_2^T d - (\|y - Ax_1\|_2^2 + \lambda x_1^T d) \\ &= \|y - Ax_2\|_2^2 - \|y - Ax_1\|_2^2 + \lambda d^T (x_2 - x_1) \\ &= \cancel{y^T y} - 2y^T A x_2 + x_2^T A^T A x_2 - \cancel{y^T y} + 2y^T A x_1 - x_1^T A^T A x_1 + \lambda d^T (x_2 - x_1) \\ &= x_2^T A^T A x_2 - x_1^T A^T A x_1 - 2y^T A (x_2 - x_1) + \lambda d^T (x_2 - x_1) \\ &\geq 2x_1^T A^T A (x_2 - x_1) - 2y^T A (x_2 - x_1) + \lambda d^T (x_2 - x_1) \\ &= (2A^T A x_1 - 2A^T y + \lambda d)^T (x_2 - x_1) \\ &= \nabla f(x_1)^T (x_2 - x_1). \end{aligned}$$

$\therefore f$ is convex in x

(h) Write the projected gradient descent update for minimizing $f(x)$ over Δ^n :

$$x^{(k+1)} = \Pi_{\Delta^n} (x^{(k)} - \eta \nabla_x f(x^{(k)})),$$

where $\eta > 0$ is a step size. Assume we run T iterations starting with the initialization $x^{(0)}$. Use algorithm unrolling to design an autoencoder that would learn A given training data. Assume η is a fixed parameter.

Soln: Using previous parts of this problem, we see that the PGD update given some $x^{(k)} \in \mathbb{R}^m$ is of the form

$$x^{(k+1)} = \Pi_{\Delta^n} (x^{(k)} - \eta (A^T (Ax^{(k)} - y) + \lambda d))$$

where $y \in \mathbb{R}$ is a data point, λ is the regularization parameter, and $d_j = \|y - a_j\|_2^2$.

Now we will use previous results to rewrite this. Note that the regularization term simply shifts the unregularized PGD over Δ^n . That is,

$$\begin{aligned} x^{(k+1)} &= \Pi_{\Delta^m} (x^{(k)} - \eta A^T (Ax^{(k)} - y) - \eta \lambda d) \\ &= \Pi_{\Delta^m} (x^{(k)} - \eta \nabla_x (\|Ax^{(k)} - y\|_2^2) - \eta \lambda d) \end{aligned}$$

We can also write this as

$$x^{(k+1)} = \Pi_{\Delta^m} ((I - \eta A^T A)x^{(k)} + \eta A^T y + \eta \lambda d)$$

So each gradient step can be modeled as follows:

Gradient-Step: Given $x^{(k)} \in \mathbb{R}^m$ and a fixed data point $y \in \mathbb{R}^d$

- (1) Transform $x^{(k)}$ by computing $\tilde{x}^{(k)} = (I - \eta A^T A)x^{(k)}$
- (2) Compute unregularized gradient $\bar{x}^{(k)} = \tilde{x}^{(k)} + \eta A^T y$
- (3) Regularize to obtain $z = \bar{x}^{(k)} + \eta \lambda d$, and return

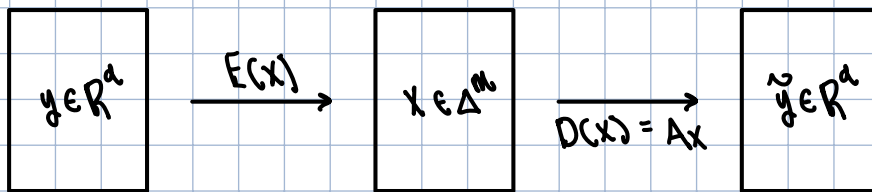
Now we want to project the result above. This is where we make use of the thresholding Interpretation in (d).

Euclidean-Projection: Given regularized gradient step $z \in \mathbb{R}^m$

- (1) Sort entries of z in decreasing order
- (2) If $z_1 - 1 \geq z_m$, go to step 8
- (3) Initialize $k = 2$
- (4) Set $S = [k]$ and compute $\tau = \frac{\sum_{i \in S} z_i - 1}{|S|}$
- (5) If $\tau \geq z_{k+1}$, go to step 7 $\rightarrow$ [Chen & Ye (2004); see example $\oplus$ in part (f).]
- (6) If $k < m$, increment k and go to step 4
- (7) If $k = m$, then set $\tau = \frac{\sum_{i=1}^m z_i - 1}{m}$
- (8) Set $\tilde{z} = \Pi_{\Delta^m}(z) = (z - \tau \mathbf{1})_+$, and return

These are the key components needed to design the autoencoder network.

The network will consist of the linear decoder A to be learned and an encoding network consisting of T PGD steps. The maps are of the form:



Want to learn $A \in \mathbb{R}^{d \times m}$ and, for each $y \in \mathbb{R}^d$, some sparse encoding $x \in \Delta^m$ st $y \approx Ax$. The autoencoder above can achieve this as follows:

- Assume training data $y_1, \dots, y_n \in \mathbb{R}^d$ and initialize $A = [a_1, \dots, a_m] \in \mathbb{R}^{d \times m}$. Since we want to approximate each y_i as a convex combination $\sum_{j=1}^m x_j a_j$, then we will need at most $d+1$ atoms for each y_i . Assuming A is overcomplete and gives the sparsity inducing properties of the PGD network, this construction is well-posed.
- Compute the extreme points of $\text{conv}\{y_i\}$, denoted P . Let $K = |P \cap \{y_i\}|$ and assume that m is chosen st $d \leq m$ and $K \leq m$. Let $a_1, \dots, a_K = P$ and, following Wu et. al.'s Δ_1 -auto encoder, initialize remaining columns of A as Gaussian random vectors. Since we want to represent y_i using convex combinations, this initialization should expedite training.
- Each block of the encoding network will include a gradient step and a Euclidean projection.
- For each data point y_i , randomly initialize $x^{(0)} \in \Delta^m$:

So the autoencoder network is given by the following algorithm:

Autoencodes: Given $y_1, \dots, y_n \in \mathbb{R}^d$

(1) Initialize $A \in \mathbb{R}^{d \times m}$ as described above

(2) For fixed y_i , initialize $x^{(0)}$ as described above

(3) Encode: For $k \in \{0, \dots, T-1\}$ DO:

 let $Z = \text{Gradient-Step}(x^{(k)}, y_i)$

 let $x^{(k+1)} = \text{Euclidean-Projection}(Z)$

(4) Decode: by computing $Ax^{(T)} = \hat{y}$

Handling (3), (4) to the layers of a neural network will allow us to train the algorithm. Thus, we can learn a dictionary A , and applying PGD with this learned dictionary (i.e. the encoder) will allow us to learn the sparse coefficients.

Homework 2 Code

Math 290 - Graduate Topics in Artificial Intelligence

Christian Lentz

```
In [64]: # import packages
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
import numpy.random as rand
import pytest
import scipy as sp
import networkx as nx
from sklearn.cluster import KMeans
```

Question 5

In this problem you will experiment with an autoencoder inspired by the model discussed in Problem 3. The first step is to generate a dataset as follows:

$$t \sim \text{Unif}(t_{\min}, t_{\max}), \quad y(t) = \begin{bmatrix} t \cos t \\ t \sin t \end{bmatrix} + \sigma \xi, \quad \xi \sim \mathcal{N}(0, I_2).$$

Set $t_{\min} = 1, t_{\max} = 10.0$ and $\sigma = 0.05$. This produces data points $y_i \in \mathbb{R}^2$ lying near a one dimensional manifold. Let $A = [a_1, \dots, a_m] \in \mathbb{R}^{2 \times m}$ be a learnable dictionary. For each data point y_i , we infer a latent code $x_i \in \Delta^m$ by minimizing

$$\min_{x_i \in \Delta^m} \|y_i - Ax_i\|_2^2 + \lambda \sum_{j=1}^m (x_i)_j \|y_i - a_j\|_2^2$$

where Δ^m denotes the probability simplex.

Inference (Encoder). For fixed A , infer x_i using projected gradient descent (PGD):

$$x_i^{(k+1)} = \Pi_{\Delta^m} \left(x_i^{(k)} - \eta \left[2A^\top (Ax_i^{(k)} - y_i) + \lambda d(y_i, A) \right] \right)$$

where $d(y_i, A) \in \mathbb{R}^m$ has entries

$$d_j(y_i, A) = \|y_i - a_j\|_2^2.$$

Dictionary Update (Decoder). For fixed codes $\{x_i\}_{i=1}^N$, update A by solving the linear system

$$A(X^\top X + \lambda \text{diag}(s) + \beta I) = (1 + \lambda)Y^\top X, \quad s := X^\top \bar{1}$$

where $X \in \mathbb{R}^{N \times m}$ stacks the codes and $Y \in \mathbb{R}^{N \times 2}$ stacks the data.

(a) Generate $N = 2000$ data points from the 2D swiss-roll model and plot them.

Solution:

```
In [19]: # global variables for this problem
N = 2000
rand.seed(1)

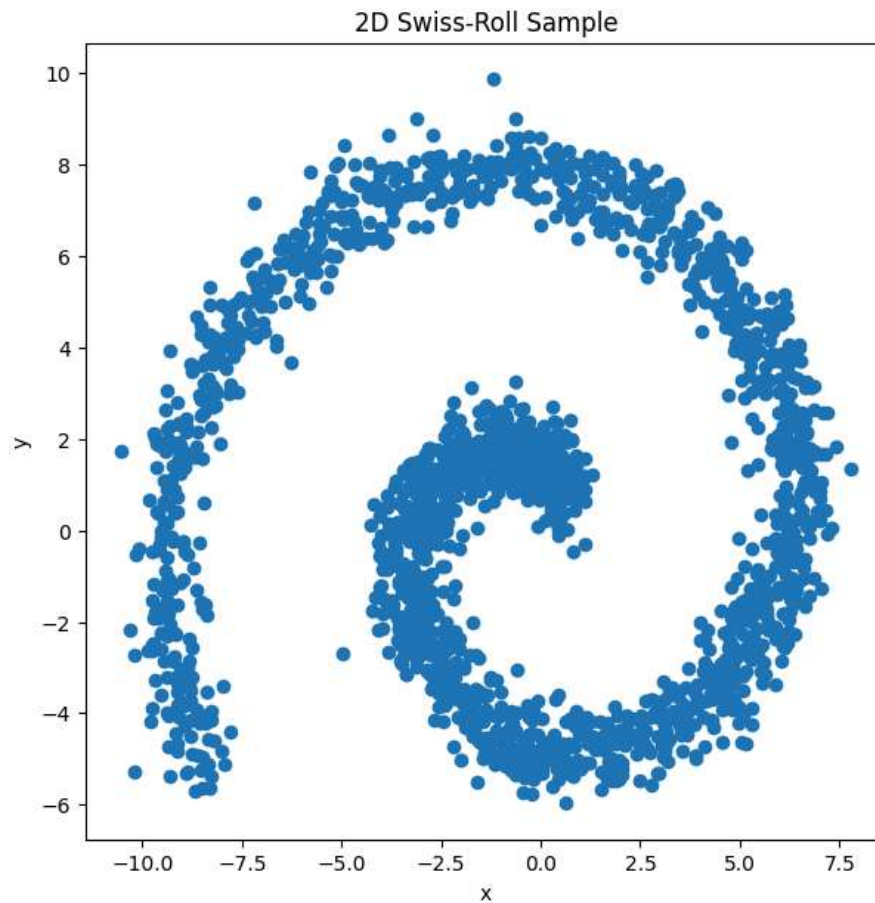
# parameters for generating data
t_min = 1.0
t_max = 10.0
sigma = 0.5

# empty (row-major) data matrix
Y = np.zeros(shape=(N,2))

# generate data
for k in range(N):
    t = rand.uniform(t_min, t_max)
    xi = rand.normal(scale=1, size=2)
    Y[k,:] = [ t*np.cos(t), t*np.sin(t) ] + sigma*xi

# plot data
# create 2D fig with matplotlib
swiss_roll_sample_2D = plt.figure(figsize=(7,7))
# extract the x and y coordinate vectors for data
xx = Y[:,0]
```

```
yy = Y[:,1]
plt.scatter(xx, yy)
# plot the result
plt.xlabel("x")
plt.ylabel("y")
plt.title("2D Swiss-Roll Sample")
plt.show()
```



(b) Implement Euclidean projection onto the probability simplex Δ^m .

Solution: The algorithm that I will implement here is detailed in question 3 of this problem set, and is inspired by the following paper: [Chen and Ye \(2024\)](#).

```
In [20]: def simplex_projection(Z):
        """
        Given a matrix `Z` with real coefficients, `m` rows, and `N` columns, project each
        column onto the probability simplex in `m` dimensional Euclidean space.

        # Parameters

        - `Z` --> a matrix with shape (m, N)

        Note that the returned matrix of simplex projections also has shape (m, N)
        """

        # sort each column of Z with entries descending
        idx = np.argsort(Z, axis=0)[::-1]
        Z_sorted = np.take_along_axis(Z, idx, axis=0)
        ## Z_sorted = np.sort(Z, axis=0)[::-1]

        # cumulatively sum over the columns of Z_sorted
        # - for example: CS[i,j] is the sum of the first i entries of the column Z[:,j]
        # - we subtract one from each cumulative sum to be in the desired form
        CS = np.cumsum(Z_sorted, axis=0) - 1

        # vector for boolean mask
        m = Z.shape[0] # number of rows
        ind = np.arange(1, m + 1).reshape(-1, 1) # indices over all rows of Z (not zero indexed)

        # boolean mask
        # - Z_sorted - (CS / ind) is a matrix, call it M
        # -  $M[i,j] = Z[i,j] - (\sum_{k=1}^i \{Z[k,j]\} - 1) / (i+1)$ 
        # - in other words, we are determining if  $Z[i,j] > (\sum_{k=1}^i \{Z[k,j]\} - 1) / (i+1)$ 
        mask = Z_sorted - (CS / ind) > 0

        # for each column of Z[:,j], determine the cutoff of the cumulative sum to compute the thresholding parameter, to
        # - find the index of the LAST active entry in each column Z[:,j]
        # NOTE: this is a vector, and each entry is an index into a column of `CS`
        i = np.sum(mask, axis=0) - 1

        # for each column, compute the thresholding parameter
```

```

# - get index i from each column of `CS` and divide by i+1
tau = CS[i, np.arange(Z.shape[1])] / (i + 1)

# project each column, UNSORT indices, and return
Z_proj_sorted = np.maximum(Z_sorted - tau, 0)
Z_proj = np.zeros_like(Z)
np.put_along_axis(Z_proj, idx, Z_proj_sorted, axis=0)

# FOR TESTING
# assert that the projection actually lies on the unit simplex
# the if statement also handles finite precision error in numpy
sums = np.linalg.matrix_transpose(Z_proj) @ np.full(m,1)
sum_of_sums = abs(np.sum(sums))
if sum_of_sums - N >= 0.0005:
    print(sum_of_sums)
    raise ValueError("The entires of the unit simplex projection must sum to one!")

return Z_proj

```

(c) Implement PGD-based inference for the latent codes x_i .

Solution: My code below implements a **single** step of the PGD, given by the following update rule:

$$x_i^{(k+1)} = \Pi_{\Delta^m} \left(x_i^{(k)} - \eta \left[2A^\top (Ax_i^{(k)} - y_i) + \lambda d(y_i, A) \right] \right)$$

where $d(y_i, A) \in \mathbb{R}^m$ has entries

$$d_j(y_i, A) = \|y_i - a_j\|_2^2$$

and $A = [a_1, \dots, a_m] \in \mathbb{R}^{2 \times m}$ is a fixed dictionary. I wrote this using a batched implementation by defining $D \in \mathbb{R}^{m \times N}$ to be the matrix such that each column vector D_{*i} has entries $D_{ji} = d_j(y_i, A)$. Therefore, the batched update becomes

$$X^{(k+1)} = \Pi_{\Delta^m} \left(X^{(k)} - \eta \left[2A^\top (AX^{(k)} - Y) + \lambda D \right] \right)$$

assuming that X, Y are provided in a column-major format and the projection is applied column-wise. My code is implemented in this manner, and transposing NumPy arrays is $O(1)$ time, so this is an efficient setup.

As was motivated in lecture, I used the Lipschitz constant of the gradient to determine the gradient step, η . This seemed to be the most beneficial choice that I made while experimenting with the model parameters. Computing this is simple. If we denote the objective as $f : \mathbb{R}^m \rightarrow \mathbb{R}$, then we can use Cauchy-Schwartz to obtain

$$\begin{aligned}\|\nabla_x f(x_1) - \nabla_x f(x_2)\|_2 &= \|2A^\top(Ax_1 - y) + \lambda d - 2A^\top(Ax_2 - y) - \lambda d\|_2 \\ &= \|2A^\top A(x_1 - x_2)\|_2 \\ &\leq 2\|A^\top A\|_2 \cdot \|x_1 - x_2\|_2\end{aligned}$$

for A a fixed dictionary, y a fixed data point, and any pair $x_1, x_2 \in \mathbb{R}^m$. So the Lipschitz constant is $2\|A^\top A\|_2$.

```
In [21]: def PGD_inference(A, Y, X, l:float):
    """
    A single step of projected gradient descent using the simplex projection.

    # Parameters

    - `A` --> a fixed dictionary with shape (2,m) and atoms along the columns.
    - `Y` --> a data matrix with shape (2,N). We want to approximate each column of this matrix as a sparse linear combination of dictionary atoms.
    - `X` --> A matrix with shape (m,N) whose columns contain the current approximation for the sparse codes of Y given dictionary A.
    - `l` --> the regularization parameter.

    # Return

    `X_next` the result of the PGD update.
    """

    # compute the distance matrix D
    # NOTE: each column D[:,j] is a vector whose entries are of the form D[i,j] = dist(Y[i,:], A[:,j])^2
    D = ( np.sum(A**2, axis=0)[:, None] + np.sum(Y**2, axis=0)[None, :] - 2 * A.T @ Y )

    # compute the gradient step --> the result is a column-major matrix with shape (m, N)
    R = (A @ X) - Y
    P = 2*(np.linalg.matrix_transpose(A) @ R)
    # use Lipschitz constant for the gradient step size (eta), as motivated in lecture
    eta = 1 / (2*np.linalg.matrix_norm(np.linalg.matrix_transpose(A) @ A, ord=2))
    G = X - eta*(P + l*D)

    # project the gradient step onto the unit simplex
```

```
PGS = simplex_projection(G)
return PGS
```

(d) Learn the dictionary A by alternating between sparse coding and dictionary learning.

Solution: Note that we will use the following lemma in my analysis:

Lemma: Let A, B be invertible matrices. Then the sum $A + B$ is an invertible matrix if, and only if, the product of the matrices commutes. That is, $AB = BA$.

This part is where our approach differs from question 3 of this problem set. Specifically, we use an alternating minimization approach here. In question 3, we unrolled the PGD to an autoencoder network in order to learn the sparse encoding map via a T step PGD. In practice, we would train this network using an algorithm such as backpropagation. From there, we could use the learned encoding map in order to learn the sparse codes via an autoencoder optimization problem. The sparsity is naturally induced via properties of the simplex projection.

The code chunks below implement the Dictionary Update (i.e. the decoding map) as well as the alternating minimization scheme.

I use the following parameters, as suggested in the remark:

- $m = 16$
- $\lambda = l = 10$
- $T = 25$, where T is the number of PGD steps
- $\beta = 10^{-2}$

Besides using a standard linear solve in SciPy, I also implemented the decoder with the following approach. In both cases, I had trouble finding parameters which adequately learned atoms with respect to the provided data.

Implementation (via matrix inversion):

Note that the decoder considers fixed codes $\{x_i\}_{i=1}^N$ and updates A by solving the linear system

$$A(X^\top X + \lambda \text{diag}(s) + \beta I) = (1 + \lambda)Y^\top X, \quad s := X^\top \mathbf{1}$$

where $X \in \mathbb{R}^{N \times m}$ stacks the codes and $Y \in \mathbb{R}^{N \times 2}$ stacks the data. It would be conceptually simple and expedite implementation if $X^\top X + \lambda \text{diag}(s) + \beta I$ were an invertible matrix. Observe that (1) $N \gg m$ in general so that $X^\top X \in \mathbb{R}^{m \times m}$ will have rank m in general and (2) βI is invertible for any choice of $\beta \neq 0$. Additionally, we see that

$$(X^\top X)(\lambda \text{diag}(s)) = (\lambda \text{diag}(s))(X^\top X)$$

since multiplication by a full-rank diagonal matrix is always commutative. So $M = X^\top X + \lambda \text{diag}(s)$ is invertible by the above lemma. We can apply the lemma once again to see that $M + \beta I$ is invertible.

So, we can solve the system to obtain A via a matrix multiplication of the form

$$A = (1 + \lambda)Y^\top X(X^\top X + \lambda(\text{diag}(s)) + \beta I)^{-1}.$$

Remark: Note that $\text{diag}(s)$ is not always guaranteed to be invertible, as it is possible for the sparse codes X to be mean centered with respect to at least one coordinate. In this case, at least one entry of $\text{diag}(s)$ would vanish, making the matrix singular. However, this occurs with incredibly low probability, especially for large N . On the other hand, assuming that we mean-center the codes X , then the term $\lambda(\text{diag}(s))$ drops off entirely, and the matrix $X^\top X + \beta I$ is always invertible for sufficiently large N . However, the caveat is that we have to mean center X each time we apply the decoding map.

```
In [22]: # NOTE: data set `Y` and its size `N` are global to this program and initialized separately.

# dictionary update / decoding
def dictionary_update(XX, m, l, beta, solve=True):
    """
    The decoding map, which we use to update the dictionary given fixed sparse codes.

    # Parameters

    - `XX` --> a fixed (row-major) matrix containing sparse codes
    - `m` --> the latent dimesnion; i.e. the number of dictionary atoms
    - `l` --> regularization parameter
    - `beta` --> dictionary update parameter
    - `solve` --> a boolean parameter determining if we use matrix inversion or a linear solve.
    """

    # first compute the matrix N
    gramX = np.linalg.matrix_transpose(XX) @ XX
    diagS = l*np.diag(np.matvec(np.linalg.matrix_transpose(XX), np.full(shape=N, fill_value=1.0)))
    I = beta*np.eye(m)
```

```

matrix_N = gramX + diagS + I

# solve via matrix multiplication
if not solve:
    # NOTE: matrix inversion is numerically unstable. Does this affect the result?
    N_inv = np.linalg.inv(matrix_N)
    YtX = (1+1)*np.linalg.matrix_transpose(Y) @ XX
    A = YtX @ N_inv
    return A

# solve via linear solve
else:
    Nt = np.linalg.matrix_transpose(matrix_N)
    XtY = (1+1)*np.linalg.matrix_transpose(XX) @ Y
    At = sp.linalg.solve(Nt, XtY)
    return np.linalg.matrix_transpose(At)

# initializing codes
def initialize_codes(m):
    """
    Randomly initialize the sparse codes.

    # Parameters

    - `m` the latent dimension; the number of dictionary atoms we are learning.
    """

    X = np.zeros(shape=(N,m))
    for k in range(N):
        x = rand.normal(scale=1, size=m)
        X[k,:] = x/np.linalg.vector_norm(x)
    return X

# minimization scheme
def alternating_minimization(A, X, m: int, T:int, l:float, beta:float, solve=True):
    """
    An alternating minimization scheme to learn a dictionary using PGD sparse coding
    and a linear solve to update the dictionary.

    # Parameters

```

```

- `A` --> the initialized dictionary with shape (2, m).
- `X` --> the initialized sparse codes with shape (N,m).
- `m` --> the latent dimension; i.e. the number of dictionary atoms to learn.
- `T` --> the number of steps to take.
- `l` --> regularization parameter.
- `beta` --> dictionary update parameter
- `solve` --> a boolean parameter determining if the decoder uses matrix inversion or a linear solve.

# The learned atoms `A` and the corresponding sparse coedes, `X`
"""

for _ in range(0,T):
    # encoding: Simplex PGD
    X_new = PGD_inference(A, np.linalg.matrix_transpose(Y), np.linalg.matrix_transpose(X), 1)
    X = np.linalg.matrix_transpose(X_new)
    # decoding: dictionary update
    A = dictionary_update(X, m, l, beta, solve)
return A, X

# the main program to learn the dictionary and plot the atoms
def learn_dictionary(m:int, l:float, T:int, beta:float, solve=True):
    """
    The main program.

    # Parameters

    - `m` --> latent dimension; i.e. the number of dictionary atoms.
    - `l` --> regularization parameter.
    - `T` --> the number of steps to take.
    - `beta` --> dictionary update parameter.
    - `solve` --> a boolean parameter determining if the decoder uses matrix inversion or a linear solve.
    """

    # randomly initialize the latent/sparse codes
    X = initialize_codes(m)
    # initialize the dictionary by running the decoder
    A = dictionary_update(X, m, l, beta, solve)
    # learn the dictionary via alternating minimization
    A_learned, X_learned = alternating_minimization(A, X, m, T, l, beta, solve)

    # plot data -----

```

```

plt.figure(figsize=(7,7))

# a vector for coloring the data
c1 = np.zeros(shape=N)
c2 = np.full(m,1)
c = np.append(c1,c2)
# extract the x and y coordinate vectors for the Learned atoms
xx_atoms = A_learned[0,:]
yy_atoms = A_learned[1,:]
# append these to the x and y coordinate vectors of the data
xx_final = np.append(xx, xx_atoms)
yy_final = np.append(yy, yy_atoms)
plt.scatter(xx_final, yy_final, c=c)
# plot the result
plt.xlabel("x")
plt.ylabel("y")
plt.title("2D Swiss-Roll Sample with Learned Atoms")
plt.show()

return A_learned, X_learned

```

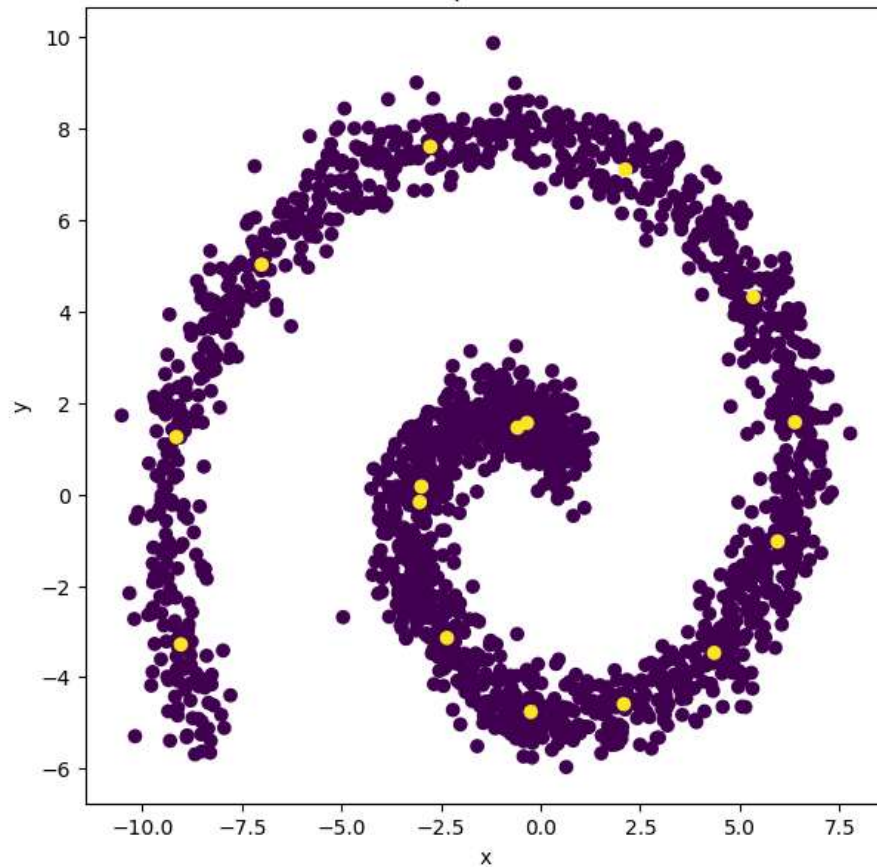
In [23]: *# the provided parameters*

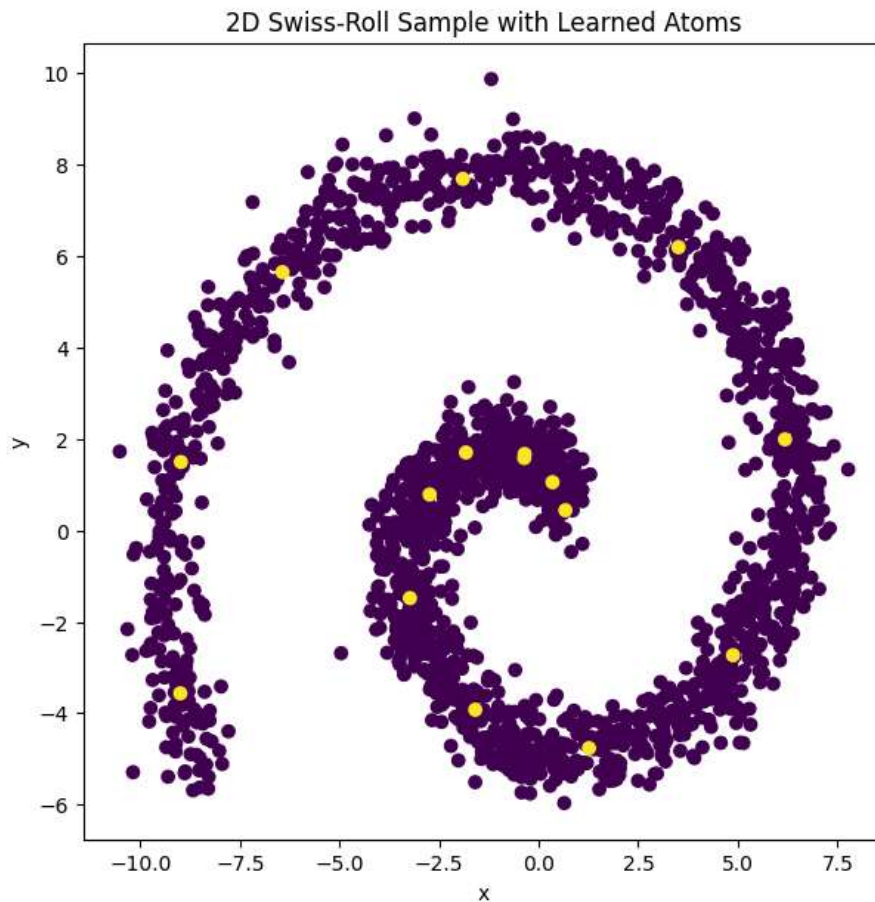
```

A, X = learn_dictionary(16,10,25,0.02) # --> with `solve`
A_not_solve, X_not_solve = learn_dictionary(16,10,25,0.02,False) # --> with matrix inversion

```

2D Swiss-Roll Sample with Learned Atoms





(e) Plot the learned atoms a_j overlaid on the data.

Solution: See the plots above.

(f) Report the final average objective value and the average support size of the latent codes $\#\{j : x_{ij} > 10^{-3}\}$.

Solution: See the code chunks below.

```
In [24]: def compute_objective(A, X, l):
        """
        Compute the objective.

        # Parameters

        - `A` --> the learned dictionary
        - `X` --> the learned codes
        - `l` --> the regularization parameter
        """

        m = A.shape[1]
        # objective
        norm = np.linalg.matrix_norm((Y.T - A @ X.T), ord='fro')
        # regularization term
        D_vec = np.array( [X[i,j]*np.dot(Y[i,:]-A[:,j],Y[i,:]-A[:,j]) for i in range(0,N) for j in range(0,m)] )
        D = D_vec.reshape(m,N,order="F")
        reg = np.linalg.matrix_norm(D, ord='fro')

        return norm + l*reg

def compute_average_sparsity(X):
    """
    Determine the average sparsity of the matrix containing the learned latent codes.

    # Parameters

    - `X` --> the learned codes
    """

    return np.sum(X > 10**(-3)) / N
```

```
In [25]: print("==== objective and sparsity (provided model parameters) ===== ")
        print("Avg objective: ", compute_objective(A, X, 10)/2000)
        print("Avg sparsity:", compute_average_sparsity(X))
        print("")
```